Hibernate

Entity

- Para que o JPA possa mapear corretamente uma classe Java para uma tabela no banco de dados utiliza-se o conceito de Entidade (Entity).
- Uma Entity deve refletir exatamente a estrutura da tabela no banco de dados para que o JPA possa tomar conta do restante do trabalho.
- Uma classe Java será considerada uma Entity ela seguir os seguintes requisitos:
- Ter a anotação @Entity
- Ter um construtor público sem parâmetros
- Ter um campo anotado como @Id

Entity

```
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;


@Entity
public class Categoria {
    @Id
    private Long id;
    private String nome;
    public Categoria(String nome) {
        super();
        this.nome = nome;
    }
    public Categoria() {
    }
    //getters e setters serão omitidos por falta de espaço

}
```

Entity

- A classe está anotada com @Entity
- Existe um atributo definido com id e anotado com @Id
- Não esqueça que a entidade deve ter um id
- Pode ser um número sequencial, mas pode ser outro tipo como String
- Não crie set para o ID, pois ele é imutável
- É obrigatório ter um construtor vazio (sem parâmetros). Outros são aceitos

Id

- Toda entidade deve ter um ID
- A JPA tem a opção de gerar um ID de modo automático
- Anotações para ID:
  - *Identity;*
  - *Sequence;*
  - *TableGenerator*.
- Cada banco adota uma solução para geração automática de id.
- Oracle e Postgres utilizam *Sequence,* SQL Server e MySQL utilizam *Identity*

Id

- Os tipos construídos/primitivos para ids simples:
  - *Byte/byte;*
  - *Integer/int;*
  - *Short/short;*
  - *Long/long;*
  - *Character/char;*
  - *String;*
  - *BigInteger;*
  - *Java.util.Date;*
  - *Java.sql.Date.*

Entity

```
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;


@Entity
@Table(name="categorias")
public class Categoria {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nome;
    public Categoria(String nome) {
        super();
        this.nome = nome;
    }
    public Categoria() {
    }

}
```

TableGenerator

A opção TableGenerator funciona da seguinte maneira:

- Uma tabela é utilizada para armazenar o valor de cada chave.
- Essa tabela contém uma coluna com o nome da tabela e outra coluna com o valor atual da chave.
- Solução que permite a portabilidade da aplicação entre banco de dados, sem a necessidade de alteração da geração de ID.
- Utilizada em qualquer banco independente de qual o tipo de geração de ID.

Entity

```
import javax.persistence.*;

@Entity
@Table(name="categorias")
public class Categoria {
    @Id
    @TableGenerator(name="TABLE_GENERATOR", table="ID_TABLE",
    pkColumnName="ID_TABLE_NAME", pkColumnValue="CATEGORIA_ID",
    valueColumnName="ID_TABLE_VALUE")
    @GeneratedValue(strategy = GenerationType.TABLE, generator="TABLE_GENERATOR")
    private Long id;
    private String nome;
    public Categoria(String nome) {
        super();
        this.nome = nome;
    }
    public Categoria() {
    }

}
```

Table Generator

Contendo a palavra:

	Tabela	Acções	Registos	Tipo	Agrupamento (Collation)	Tamanho	Suspenso
☐	categorias	Procurar Estrutura Pesquisar Insere Limpa Elimina	1	InnoDB	latin1_swedish_ci	16.0 KB	-
☐	id_table	Procurar Estrutura Pesquisar Insere Limpa Elimina	1	InnoDB	latin1_swedish_ci	16.0 KB	-
☐	produto	Procurar Estrutura Pesquisar Insere Limpa Elimina	1	InnoDB	latin1_swedish_ci	32.0 KB	-
	3 tabelas	Soma	3	InnoDB	latin1_swedish_ci	64.0 KB	0 Bytes

TableGenerator

- A JPA cria uma tabela no BD
- *name* - É o id do TableGenerator dentro da aplicação.
- *table* - Nome da tabela que irá conter as chaves de cada tabela.
- *pkColumnName* - Nome da coluna que irá conter o nome de id. No código acima a coluna com o nome id_table_name será gerada.
- *valueColumnName* - Nome da coluna que irá conter os valores de cada id.
- *pkColumnValue* - Nome da tabela que será salvo. O valor default é o nome da classe + id. Em nosso caso, é CATEGORIA_ID.
- É possível declarar o TableGenerator em outras entidades sem afetar o sequencial de uma entidade como visto com Sequence.
- A melhor prática é utilizar o TableGenerator dentro do arquivo orm.xml, utilizado para sobrescrever as configurações do JPA.

Id

- O controle do ID é responsabilidade do BD, a JPA não interfere nisto.
- É necessário a persistência do objeto e depois uma consulta para descobrir o ID do objeto.
- Há perda de desempenho

Utilizando Chave Composta

- Chaves compostas podem ser simples ou complexas
- Ela contém mais de um atributo para ser o identificador da entidade

- Duas soluções @IdClass e @Embeddable

```
@Entity
@IdClass(ProdutoSeriadoId.class)
public class ProdutoSeriado {
	@Id
	private long id;
	@Id
	private int serial;
	private String nome;
	public ProdutoSeriado(String nome, String descricao,
BigDecimal preco, Categoria categoria) {
		super();
		this.nome = nome;
	}
	public ProdutoSeriado() {
		super();
	}
}
```

```java
public class ProdutoSeriadoId implements Serializable{
    private static final long serialVersionUID = 22L;

    private long id;
    private int serial;

  public ProdutoSeriadoId() {
  }

  public ProdutoSeriadoId (long id, int serial) {
    this.serial = serial;
    this.id = id;
  }
```

```java
    // Deve ter um método hashCode
    @Override
    public int hashCode() {
        final int prime = 31;
        int hash = 17;
        hash = (int) (hash * prime + this.id);
        hash = hash * prime + this.serial;
        return hash;
    }

    // Deve ter um método equals
    @Override
    public boolean equals(Object obj) {
        if (obj == this)
                return true;
        if ( ! (obj instanceof ProdutoSeriadoId))
                return false;
        ProdutoSeriadoId other = (ProdutoSeriadoId) obj;
        return (this.id == other.id ) && (this.serial == other.serial);
    }
}
```

@IdClass

- @IdClass(ProdutoSeriadoId.class) -> Essa anotação indica que a classe ProdutoSeriadoId contém os campos considerados com Id.
- Todos os campos que são marcados com Id devem estar descritos na classe ProdutoSeriadoId.
- É possível também utilizar @GeneratedValue na chave composta neste tipo de chave composta. No exemplo acima é possível adicionar @GeneratedValue ao atributo serial.

@IdClass

- Deve ter um construtor vazio;
- Implementar a interface Seriazable;
- Sobrescrever os métodos hashCode e equals

@EmbeddedId

```
@Entity
public class Car {

  @EmbeddedId
  private CarId carId;
  private String nome;
  public String getName() {
    return nome;
  }
  public CarId getCarId() {
    return carId;}
```

@Embeddable

```
import javax.persistence.Embeddable;


@Embeddable
public class CarId implements Serializable{
private static final long serialVersionUID = 23L;

private long id;
private int serial;
public CarId() {
super();
}
```

Associação One to One Unidirecional

```java
@Entity (name="armario")
public class Armario {
    @Id
    @GeneratedValue
    private int id;
    private int corredor;
    private int prateleira;
    private int number;
    public int getId() {
      return id;
    }
    public Armario() {
     super();
    }

    public Armario(int corredor, int prateleira, int number)
    {
        super();
        this.corredor = corredor;
        this.prateleira = prateleira;
        this.number = number;
    }
```

```java
@Entity
public class Pessoa {
@Id
@GeneratedValue
   private int id;
   private String nome;
   @OneToOne
   @JoinColumn(name="armario_id")
   private Armario armario;


   public Pessoa() {
      super();
   }


   public Pessoa(String nome) {
      super();
      this.nome = nome;
   }
```

Associação One to One Bidirecional

```
@Entity (name="armario")
public class Armario {
@Id
@GeneratedValue
   private int id;
   private int corredor;
   private int prateleira;
   private int number;

   @OneToOne(mappedBy="armario")
   private Pessoa pessoa;

   public int getId() {
      return id;
   }
   public Armario() {
      super();
   }
```

```java
@Entity
public class Pessoa {
@Id
@GeneratedValue
   private int id;
   private String nome;
   @OneToOne
   @JoinColumn(name="armario_id")
   private Armario armario;


   public Pessoa() {
      super();
   }


   public Pessoa(String nome) {
      super();
      this.nome = nome;
   }
```

Associação One to Many Unidirecional

```java
@Entity
@Table(name="categoria")
public class Categoria {
  @Id
  @GeneratedValue(strategy =
  GenerationType.IDENTITY)
  private Long id;
  private String nome;
  public Categoria(String nome) {
    super();
    this.nome = nome;
  }
}
```

```java
@Entity
@Table(name = "produto")
public class Produto {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private long id;
    private String nome;
    private String descricao;
    private BigDecimal preco;
    private LocalDate dataCadastro = LocalDate.now();
    //relacionamento um para muitos_o produto leva o id da categoria
    @ManyToOne
    private Categoria categoria;
    public Produto(String nome, String descricao, BigDecimal preco, Categoria
    categoria) {
        super();
        this.nome = nome;
        this.descricao = descricao;
        this.preco = preco;
        this.categoria = categoria;
        }
        //faltam getter e setters
}
```

Associação One to Many Bidirecional

```
@Entity
@Table(name = "produto")
public class Produto {
      @Id
      @GeneratedValue(strategy = GenerationType.IDENTITY)
      private long id;

      private LocalDate dataCadastro = LocalDate.now();
      //relacionamento um para muitos  produto leva o id da categoria
      @OneToOne (mappedBy=categoria”)
      private Categoria categoria;
      public Produto( BigDecimal preco, Categoria categoria) {
            super();
                  }
      public Produto() {
            super();
      }
//Getters e Setters
}
```

```
@Entity
@Table(name="categoria")
public class Categoria {
  @Id
  @GeneratedValue(strategy =
  GenerationType.IDENTITY)
  private Long id;
  private String nome;
  public Categoria(String nome) {
    super();
    this.nome = nome;
  }
  @OneToMany(mappedBy = ”categoria")
  private List<Produto> produtos;
```

Associação Many to Many Bidirecional

```
@Entity
public class Dono {
   @Id
   @GeneratedValue
   private int id;
   private String nome;

   @ManyToMany
   @JoinTable(name="dono_cao",joinColumns =
   @JoinColumn(name="dono_id"),
   inverseJoinColumns = @JoinColumn(name= "cao_id"))
   private List<Cao> matilha;


}
```

```java
@Entity
public class Cao {

  @Id
  @GeneratedValue
  private int id;
  private String nome;

  @ManyToMany(mappedBy="matilha")
  private List<Dono> familia;

}
```

```java
public class DonoCao {

  @Id
  @ManyToOne
  @JoinColumn(name="dono_id")
  private Dono dono;
  @Id
  @ManyToOne
  @JoinColumn(name="cao_id")
  private Cao cao;

  @Temporal(TemporalType.DATE)
  private Date adocaoData;
  }
```